Guia Essencial de Quality Assurance (QA)

1. O que é QA (Quality Assurance)

Quality Assurance é a área responsável por garantir a qualidade de um software **desde a concepção até a entrega e manutenção**. O foco do QA não é apenas encontrar defeitos, mas **prevenir erros**, melhorando processos, comunicação e entendimento do produto.

Resumo: QA garante que o sistema seja construído da forma correta.

---

2. QA x QC (Quality Control)

QA – Garantia da Qualidade - Foco no processo - Atua desde o início do projeto - Define padrões e boas práticas - Previne defeitos - Trabalha junto ao time inteiro

QC – Controle da Qualidade (Tester) - Foco no produto pronto - Executa testes manuais e automatizados - Encontra defeitos existentes

Perguntas-chave: - QA: *Estamos fazendo do jeito certo?* - QC: *O que foi feito funciona?*

---

3. Papel do QA no Ciclo de Desenvolvimento

O QA participa de todas as etapas: - Requisitos - Planejamento - Desenvolvimento - Testes - Entrega - Pós-entrega

Quanto mais cedo um erro é identificado, **menor o custo para a empresa**.

---

4. Requisitos e Regras de Negócio

Requisitos Funcionais: O que o sistema faz. > Ex: O sistema deve permitir login.

Requisitos Não Funcionais: Como o sistema se comporta. > Ex: O login deve responder em até 2 segundos.

Regras de Negócio: Condições obrigatórias do domínio. > Ex: Um pagamento via PIX expira em 15 minutos.

---

5. Metodologias Ágeis (Scrum e Kanban)

Épico: Grande objetivo do sistema.

História de Usuário: Funcionalidade com valor para o usuário. > Como cliente, quero pagar com PIX para finalizar a compra rapidamente.

Task (Tarefa): Trabalho técnico necessário para concluir a história.

---

6. Níveis de Teste

1. **Teste Unitário** – feito pelo desenvolvedor (funções isoladas)
2. **Teste de Integração** – comunicação entre módulos
3. **Teste de Sistema** – sistema completo (principal atuação do QA)
4. **Teste de Aceitação** – validação final pelo PO ou cliente

---

7. Tipos de Teste (nível de sistema)

- **Funcional**: verifica se funciona conforme o requisito
  - *Exemplo:* validar se o login aceita usuário e senha válidos.
- **Regressão**: garante que funcionalidades antigas não quebraram
  - *Exemplo:* após corrigir o login, testar compra, cadastro e perfil.
- **Usabilidade**: avalia facilidade de uso
  - *Exemplo:* verificar se o botão de pagamento é visível e intuitivo.
- **Performance**: mede tempo de resposta, carga e estresse
  - *Exemplo:* API de login deve responder em até 2 segundos com 5 mil usuários.
- **Segurança**: protege dados e acessos
  - *Exemplo:* tentar acessar endpoint sem token JWT.
- **Exploratório**: testes livres baseados em experiência
  - *Exemplo:* clicar rapidamente, usar dados inválidos, trocar abas.

Regra prática: > Teste funcional diz se funciona. Teste não funcional diz se presta.

---

8. Caixa-Preta x Caixa-Branca

Caixa-Branca: - Analisa o código - Geralmente feita pelo desenvolvedor

Caixa-Preta: - Analisa comportamento - Não exige conhecimento do código - Principal abordagem do QA

---

9. Erro, Defeito e Bug (conceito fundamental)

Entender a diferença entre erro, defeito e bug é essencial para comunicação profissional em QA.

Erro (Error)

Definição: Erro é uma falha humana. Ocorre quando alguém interpreta, decide ou executa algo de forma incorreta.

Onde ocorre: - Pessoas (PO, Dev, QA, Stakeholders)

Exemplo: O PO informa que o pagamento via PIX expira em 30 minutos, quando a regra correta é 15 minutos.

Defeito (Defect)

Definição: Defeito é o erro humano materializado no sistema, como código, configuração ou regra implementada incorretamente.

Onde ocorre: - Código - API - Banco de dados - Configurações do sistema

Exemplo: O código é implementado permitindo pagamento via PIX após 30 minutos.

Bug

Definição: Bug é o defeito percebido durante a execução do sistema, causando comportamento incorreto visível.

Onde ocorre: - Durante testes - Em homologação - Em produção

Exemplo: O usuário consegue finalizar um pagamento via PIX após o tempo limite permitido.

Linha do tempo do problema

Erro (humano) → Defeito (sistema) → Bug (comportamento)

Visão do QA

- O QA atua para **evitar erros** analisando requisitos

- O QA busca **identificar defeitos** antes da liberação
- O QA reporta **bugs** quando o sistema apresenta falhas

---

10. Técnicas de Teste

10. Artefatos de Teste

- **Cenário de Teste**: situação a ser validada
- **Caso de Teste**: passo a passo com resultado esperado
- **Caso de Uso**: documentação formal do fluxo

---

11. BDD e Gherkin

BDD (Behavior Driven Development) descreve o comportamento do sistema em linguagem de negócio.

Estrutura: - Dado que... - Quando... - Então...

---

12. Automação de Testes

A automação é usada principalmente para: - Regressão - Smoke tests - Fluxos críticos - Testes repetitivos

Automação com Playwright / Cypress (Front-end)

Exemplo (login automatizado): - Abrir a aplicação - Preencher usuário e senha - Clicar em login - Validar redirecionamento e mensagem

Esses testes simulam o comportamento real do usuário (E2E).

Automação de API

Usada para validar regras de negócio sem depender da interface.

Exemplo: - Enviar POST /login - Validar status 200 - Validar token retornado - Validar mensagem de erro em credenciais inválidas

Ferramentas comuns: - Playwright (APIRequest) - Cypress (cy.request) - Postman / Newman

Automação com Banco de Dados

Usada para validar persistência dos dados.

Exemplo: - Executar pagamento - Consultar banco - Verificar se pedido foi criado com status correto

Automação **não substitui o QA**, ela aumenta sua eficiência.

---

13. Bug Report (Relato de Defeitos)

Um bom bug report contém: - Título claro - Descrição objetiva - Passos para reproduzir - Resultado atual - Resultado esperado - Ambiente - Evidências - Severidade - Prioridade

Regra de ouro: > Se o desenvolvedor não consegue reproduzir, o bug não existe.

---

14. Ambientes x Fases (conceito essencial)

Fase

É o **momento do processo** de desenvolvimento.

Exemplos de fases: - Desenvolvimento - Testes - Homologação - Produção

A fase responde à pergunta: > *Em que etapa do processo estamos?*

Ambiente

É o **local técnico** onde o sistema está rodando.

Exemplos de ambientes: - Dev - QA / Testes - Homologação (HML / Staging) - Produção (Prod)

O ambiente responde à pergunta: > *Onde o sistema está rodando?*

Relação prática

- Uma **fase** pode usar mais de um ambiente
- Um **ambiente** pode existir durante várias fases

Exemplo: Durante a fase de testes, o QA usa o ambiente de QA. Durante a fase de homologação, o PO usa o ambiente de homologação.

---

15. Automação por Ambiente (como empresas fazem)

Ambiente de Testes (QA)

Objetivo: encontrar bugs

Automações comuns: - Testes E2E com Playwright ou Cypress - Testes de API - Testes de regressão

Exemplo: - Fluxo completo de login e pagamento

Ambiente de Homologação

Objetivo: validar negócio

Automações comuns: - Smoke tests - Testes críticos automatizados

Exemplo: - Verificar se o sistema subiu corretamente - Garantir que pagamento e login funcionam

---

Ambiente de Produção

Objetivo: operar o sistema

Automação permitida: - Monitoramento - Health check - Alertas

⚠️ Testes funcionais **não devem ser executados em produção**

---

16. Critérios de Liberação (Go / No-Go)

Antes de liberar para produção, o time avalia:

Go (liberar)

- Testes críticos passaram
- Nenhum bug crítico aberto
- Regressão executada
- PO aprovou a entrega

No-Go (não liberar)

- Bug crítico ou alto impacto
- Regressão falhou
- Regra de negócio não validada

QA participa ativamente da decisão, apresentando riscos.

---

17. Visão Geral do Fluxo de Qualidade

Requisitos → Regras de Negócio → Histórias → Cenários → Casos de Teste / BDD → Testes → Automação → Regressão → Homologação → Go/No-Go → Produção

---

18. Princípio Final

> Qualidade não é responsabilidade apenas do QA, mas de todo o time. O QA garante que isso aconteça.

Perfeito. Vou deixar **mais sucinto, direto e organizado**, mas ainda completo o suficiente para ocupar aproximadamente **duas páginas de apostila**.

Scrum e Kanban: Fundamentos para QA

1. Introdução às Metodologias Ágeis

As metodologias ágeis surgiram como resposta aos modelos tradicionais de desenvolvimento, que eram mais rígidos e baseados em planejamento extensivo. O Manifesto Ágil (2001) prioriza:

- Indivíduos e interações
- Software em funcionamento
- Colaboração com o cliente
- Resposta a mudanças

Entre os principais frameworks utilizados estão **Scrum** e **Kanban**, amplamente adotados em equipes de desenvolvimento e qualidade de software (QA).

---

2. Scrum

2.1 Conceito

Scrum é um framework ágil baseado em ciclos curtos chamados **Sprints** (1 a 4 semanas). A cada Sprint, a equipe entrega um **incremento funcional** do produto.

O foco é:

- Entrega contínua de valor
- Transparência
- Inspeção e adaptação

2.2 Papéis no Scrum

Product Owner (PO)

- Define e prioriza o **Product Backlog**.
- Representa os interesses do cliente e stakeholders.
- Define critérios de aceitação.
- Aceita ou rejeita as entregas.

Scrum Master

- Garante a aplicação correta do Scrum.

- Remove impedimentos.
- Facilita cerimônias.
- Promove melhoria contínua.

Time de Desenvolvimento

- Equipe multidisciplinar (Dev, QA, UX etc.).
- Autogerenciável.
- Responsável por entregar o incremento.

O QA faz parte do Time de Desenvolvimento.

2.3 Eventos do Scrum

- **Sprint Planning**: definição do que será desenvolvido na Sprint.
- **Daily Scrum**: reunião diária de alinhamento.
- **Sprint Review**: apresentação do que foi entregue.
- **Sprint Retrospective**: análise interna para melhoria do processo.

2.4 Artefatos

Product Backlog

Lista priorizada de funcionalidades, melhorias e correções.

Sprint Backlog

Itens selecionados para a Sprint atual.

Incremento

Resultado funcional entregue ao final da Sprint.

---

3. Estrutura dos Itens de Trabalho

3.1 Épico

Grande funcionalidade ou objetivo estratégico que pode ser dividido em várias histórias.

Exemplo:
Implementar sistema de autenticação completo.

3.2 História de Usuário

Descrição de uma funcionalidade sob a perspectiva do usuário.

Formato padrão:

Como [usuário],
Quero [funcionalidade],
Para [benefício].

Exemplo:

Como cliente, quero redefinir minha senha para recuperar o acesso à minha conta.

Deve conter:

- Clareza
- Valor de negócio
- Critérios de aceitação
- Testabilidade

3.3 Task (Tarefa)

Divisão técnica da história de usuário.

Exemplos:

- Criar endpoint de redefinição.
- Implementar validação de token.
- Criar testes automatizados.
- Executar testes funcionais.

Tasks organizam a execução técnica da história.

---

4. Kanban

4.1 Conceito

Kanban é um método de gestão visual focado em fluxo contínuo e melhoria incremental. Não trabalha com Sprints fixas.

Objetivos:

- Visualizar o trabalho
- Reduzir gargalos
- Melhorar fluxo
- Limitar trabalho em progresso (WIP)

4.2 Quadro Kanban

Organizado em colunas como:

- Backlog
- A Fazer
- Em Desenvolvimento
- Em Teste
- Concluído

Cada tarefa se movimenta conforme o progresso.

4.3 Limite de WIP

O Kanban estabelece limites de trabalho simultâneo para evitar sobrecarga e aumentar previsibilidade.

Exemplo:
Máximo de 3 tarefas em desenvolvimento por vez.

---

5. Atuação do QA em Scrum e Kanban

No Scrum

- Participa da Sprint Planning.
- Auxilia na definição de critérios de aceitação.
- Elabora casos de teste.
- Executa testes funcionais, exploratórios e automatizados.

- Valida a entrega antes da Review.
- Participa da Retrospectiva propondo melhorias.

O QA atua de forma preventiva e não apenas corretiva.

No Kanban

- Atua na validação contínua.
- Identifica gargalos na coluna de testes.
- Monitora qualidade do fluxo.
- Sugere melhorias no processo.
- Trabalha com entrega contínua.